

JC4003 Natural Language Processing

Lecture 11: Bert

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

Pre-trained NLP models

BERT

- Architecture
- Training
- Fine-tune

BERT in NLP tasks

BERT's Family

BERT

Devlin, J., et al. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Pre-trained NLP Models

Pre-trained NLP models are language models that have been trained on large corpora.

By learning from vast amounts of language data, these models develop a general understanding of language, which can then be fine-tuned for specific NLP tasks like text classification, sentiment analysis, or machine translation.

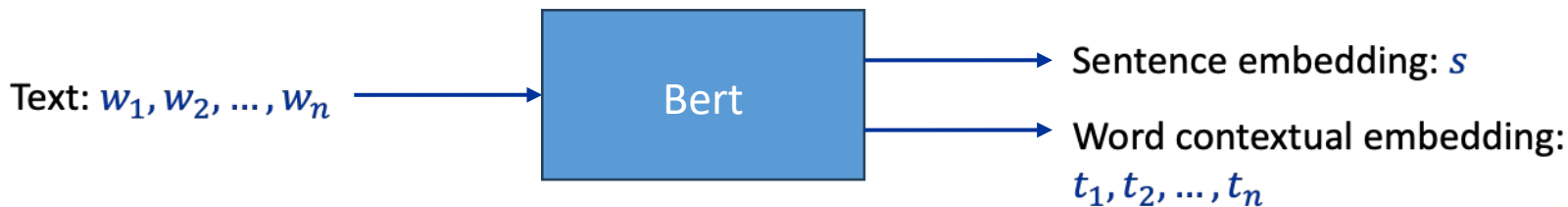
Popular pre-trained models include:

- Word2Vec, GloVe: captures static word embeddings
- BERT, RoBERTa, etc.: captures context from both directions in a text.
- GPT: for text generation, which predicts the next word in a sequence.
- T5: A model that treats every NLP task as a text-to-text problem.
- ELMo: A model that generates context-sensitive word embeddings.

BERT Architecture

BERT (Bidirectional Encoder Representations from Transformers), developed by Google, is designed to capture **bidirectional context**. This allows BERT to consider both the preceding and following words in a sentence, generating **embeddings for each token (i.e. word)** as well as a **contextual semantic embedding that represents the entire input sentence**.

BERT's structure is very simple, **based purely on stacked Transformer encoders**. BERT comes in two versions: one with **12 layers** and another with **24 layers**.



WordPiece Tokenisation

Like all Transformers, BERT takes **text as input**, such as a sentence. However, instead of using embeddings for each whole word, BERT breaks words down into smaller subword units (i.e. **WordPiece**) and uses the embeddings for them.

BERT's tokenizer applies WordPiece tokenisation **to handle out-of-vocabulary words or rare words**. If a word is not in the model's vocabulary, it is broken **down into smaller subword units or even individual characters**.

unhappiness → **un**, **##happy**, **##ness**

The **## prefix** presents a part of the token, indicating the token is part of a word and not a standalone token.

WordPiece Tokenisation (cont.)

BERT introduces **special tokens into the text** to represent specific meanings or functions.

BERT adds special tokens to the sequence, e.g.,

[CLS] the deadline for the nl ##p project is approaching . [SEP]

- [CLS] is added at the beginning of the text, serving as a marker for classification tasks.
- [SEP] is added at the end of the text (or between sentences in sentence-pair tasks during training, marking the separation between different parts of the input.)

Once the text is tokenised into subwords, **each token is mapped to a unique ID from BERT's vocabulary**. These token IDs are then used as input to the model.

BERT's Embedding

BERT's embedding is composed of three different types of embeddings that are added together:

Token Embeddings (x_t): These are the standard embeddings for words or subwords, where each token is mapped to a fixed-dimensional vector.

Positional Embeddings (x_p): positional embeddings provide information about the position of each token in the sequence. BERT uses **Learned Position Embeddings**.

Segment Embeddings (x_s): used for training BERT, and in most cases, it is fine to use their default values (i.e. 0) when applying the model.

The final embedding (x) for each token in BERT is **the sum** of its token embedding:

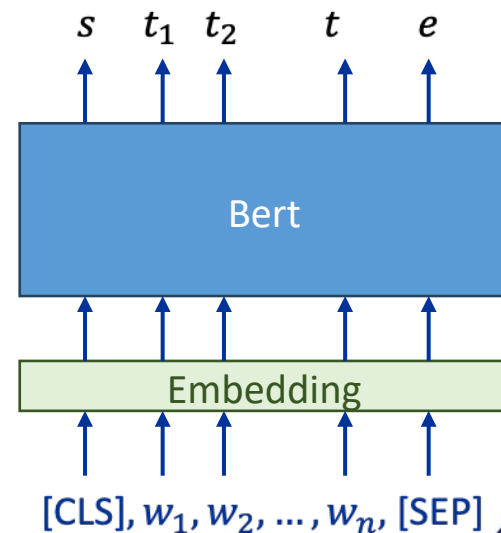
$$x = x_t + x_p + x_s$$

BERT's Output

The output of BERT, which is the output of the Transformer for each token (per-token), can be divided into two categories: the output for each word/subword and the output for each special token i.e. [CLS] and [SEP].

Per-token Embeddings (t_i): BERT generates an embedding for each word/subword. It represents the **contextualised meaning** of each word/subword i.e. the semantics of a token based on the entire surrounding context (unlike Word2Vec does).

[CLS] Token Embedding (s): This embedding represents the semantics of the **entire sequence**.



BERT's Training

BERT's Training

From BERT's output, we can see that the **output at each position serves a significantly different function**. By examining BERT's training process, we can understand how the training forces BERT's output to develop in this way.

BERT's training involves two self-supervised objectives:

- **Masked Language Modelling (MLM)**: to **predict randomly masked words** in input sentences based on their surrounding context.
- **Next Sentence Prediction (NSP)**: to understand the relationship between two input sentences by predicting **whether the second sentence logically follows the first one**.

This training equips BERT with a strong understanding of language so that BERT can then be fine-tuned for various downstream NLP tasks.

BERT's Training - Masked Language Modelling

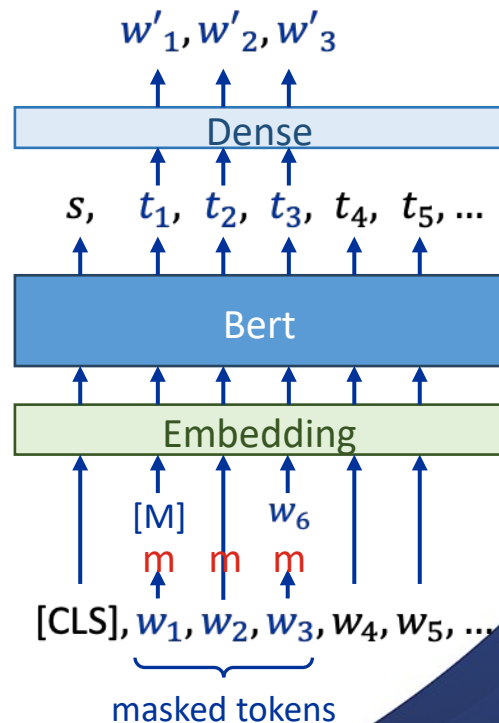
Unlike traditional language models, BERT is trained to predict randomly masked words within a sentence.

During training, 15% of the tokens in each input sequence are randomly selected for **masking**. Of those selected tokens, 80% are replaced with the **[MASK]** token, 10% are replaced with a **random word**, and 10% are left **unchanged**.

The model is then tasked with predicting the original tokens that were masked, based on the surrounding context. E.g. when w_1, w_2, w_3 are masked, t_1, t_2, t_3 will be in loss:

$$\mathcal{L} = CE(W \cdot t_1, w_1) + CE(W \cdot t_2, w_2) + CE(W \cdot t_3, w_3)$$

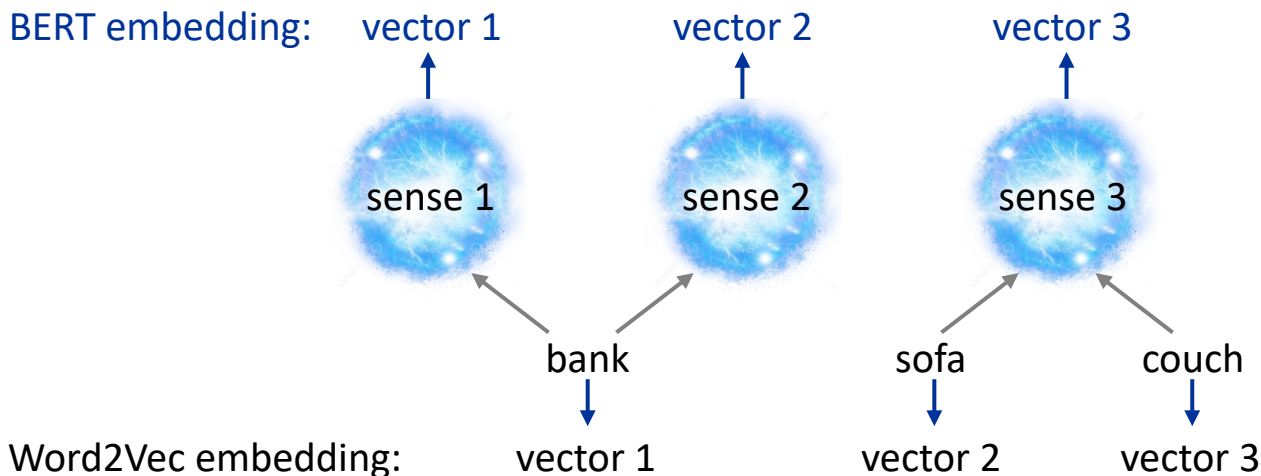
where W is the parameters in an additional prediction layer.



Masked Language Modelling

Masked Language Modelling forces BERT to recover tokens according to their surrounding words. Therefore, unlike Word2Vec, per-token embeddings capture the **meaning of a word in its context**, rather than the meaning of the word itself.

For example, the embedding for the word "bank" would be **different** depending on whether it appears in the context of "financial bank" or "river bank".



Masked Language Modelling (impact)

Masked Language Modelling (MLM) has had a significant impact on subsequent work:

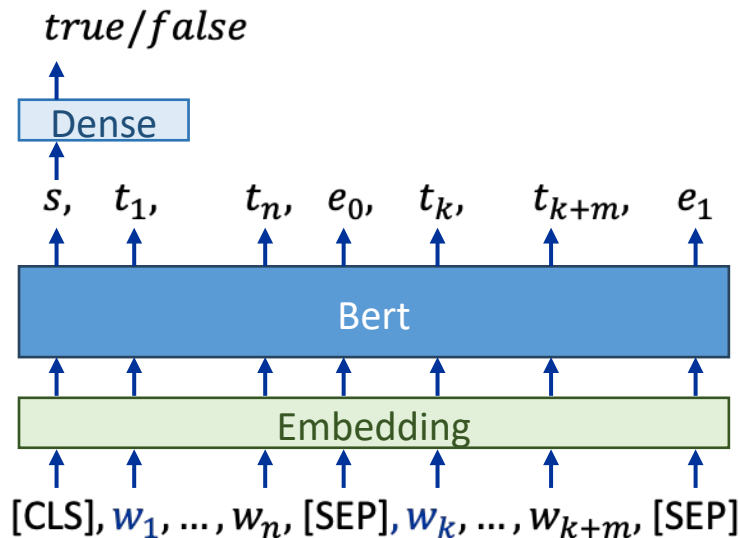
Bidirectional Context Understanding: MLM showed that to understand both the left and right context of words could result in more powerful language representations. This inspired many other models to adopt or build upon bidirectional training methods.

Improved Transfer Learning: MLM led to widespread adoption of subsequent NLP models. This approach has **become a standard paradigm in NLP**, significantly improving performance across tasks like text classification, question answering, named entity recognition, etc.

BERT's Training - Next Sentence Prediction

During training, BERT's input is a **pair of sentences** rather than a single sentence. These two sentences are separated by [SEP] token. The model is trained to **predict whether the second sentence logically follows the first one or not**.

During training, for 50% of the pairs, the second sentence is the actual next sentence, while for the other 50%, it is a randomly selected sentence from the corpus.



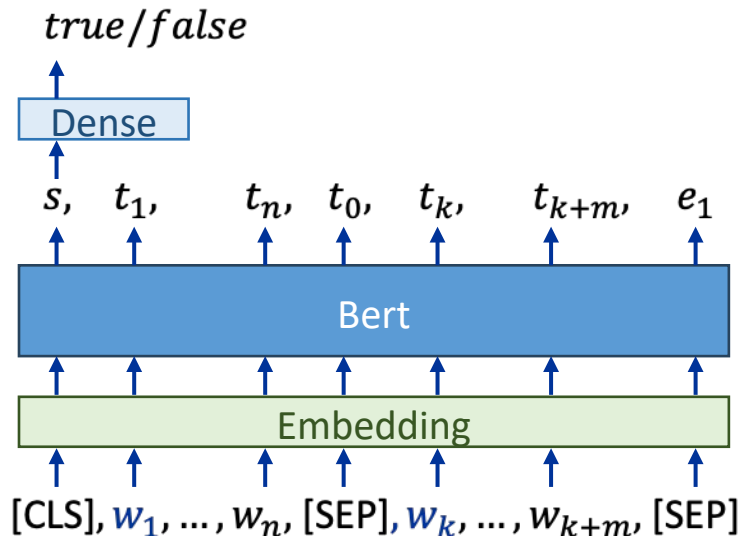
BERT's Training - Next Sentence Prediction

Segment embeddings have only two possible values (denoted by v_0 and v_1), used to differentiate between BERT's two sentence inputs (v_0 for every token in the first sentence, and v_1 for the second sentence).

The [CLS] embedding (s) is used to make a prediction by passing s through an additional dense layer, which outputs is the binary prediction.

$$\mathcal{L} = CE(W \cdot s, c)$$

where W is the parameters in an additional prediction layer, c is the binary label indicating the sentence relationship.



[CLS] Embedding vs. Sentence Semantics

The NSP task alone may not be sufficient for ensuring the [CLS] embedding captures the sentence-level semantics; however, when combined with the MLM task, the [CLS] embedding does.

The MLM ensures that BERT's output contains **contextual semantics**, and since the [CLS] embedding is part of the output, it also includes this contextual information. Additionally, the NSP task ensures that the [CLS] embedding contains the **necessary information for distinguishing between sentence pairs**.

Therefore, under the combined influence of **MLM and NSP**, the [CLS] embedding captures the **contextual semantics required to differentiate between sentences**, which is the **global semantic information of the sentence**.

Next Sentence Prediction (impact)

The NSP task demonstrated the potential of modelling sentence-level relationships during pre-training, enabling models to understand not only the relationships between words but also the logical connections between sentences.

It is particularly useful for tasks that require cross-sentence understanding, such as natural language inference (NLI) and sentence similarity assessments.

This capability has proven valuable for tasks such as question answering and textual inference, encouraging further research into improving cross-sentence relationship modelling.

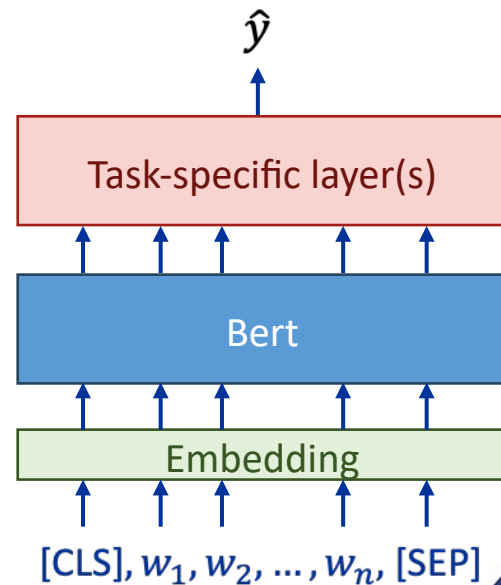
Fine-tuning

Fine-tuning

Fine-tuning is the process of taking the pre-trained BERT (or other pre-trained model) and **further training it** on a specific task or dataset to adapt it to the requirements of that task while **leveraging the general knowledge the BERT has already acquired**.

To fine-tune BERT, you:

1. **add task-specific layers on top**, passing the BERT's outputs to the layers and use the layer's outputs as the model output
2. load the learnt weights from the pre-trained model
3. train the entire model (or just the new layers) on the task-specific dataset to adjust the entire model to the new task



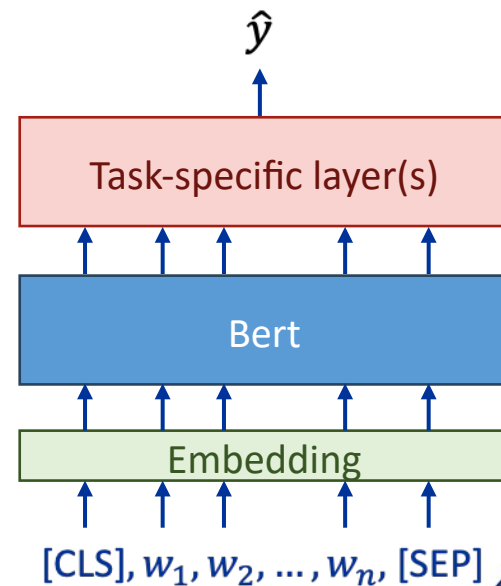
Fine-tuning (cont.)

When fine-tuning BERT, if the **target task's dataset is small** or the **training time is too long**, it may lead to **overfitting** or **catastrophic forgetting**, which can undermine the knowledge BERT acquired during pre-training.

When fine-tuning BERT, it is usually necessary to employ the following techniques:

- use a smaller learning rate (such as $1e-5$ to $5e-5$),
- apply gradient clipping, and
- use stronger regularisation methods like Dropout.

If needed, you can freeze BERT's weights and train only the task-specific layer, or gradually unfreeze BERT's layers during training.

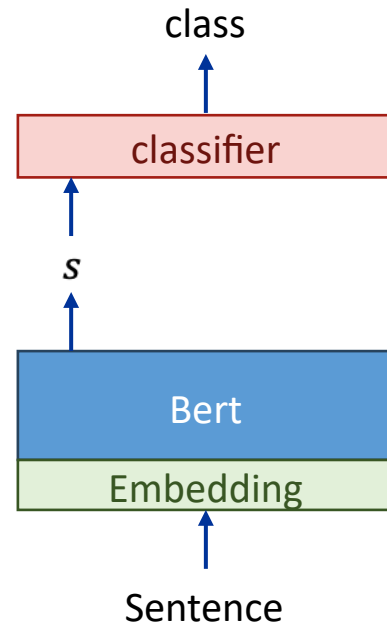


The Application of BERT in NLP Tasks

Text Classification

Sentiment analysis, spam detection, news categorisation, etc.

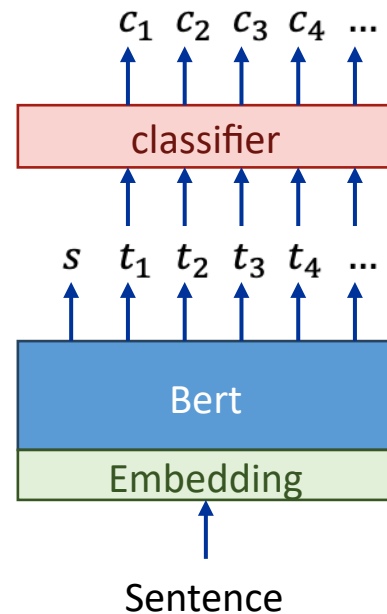
BERT is fine-tuned for text classification by using the [CLS] token embedding (which represents the entire input sequence) as the input to a classifier that predicts the category of the text.



Named Entity Recognition (NER)

Identifying entities like names, dates, locations, and organisations in text.

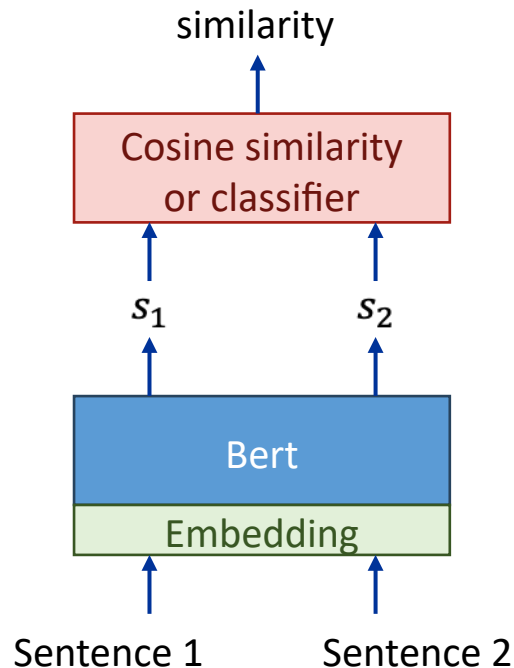
BERT generates **token-level embeddings** for each word, which are then used to classify each word into different entity categories respectively.



Sentence Similarity

Determining whether two sentences have **similar meanings**, such as in semantic textual similarity (STS) tasks.

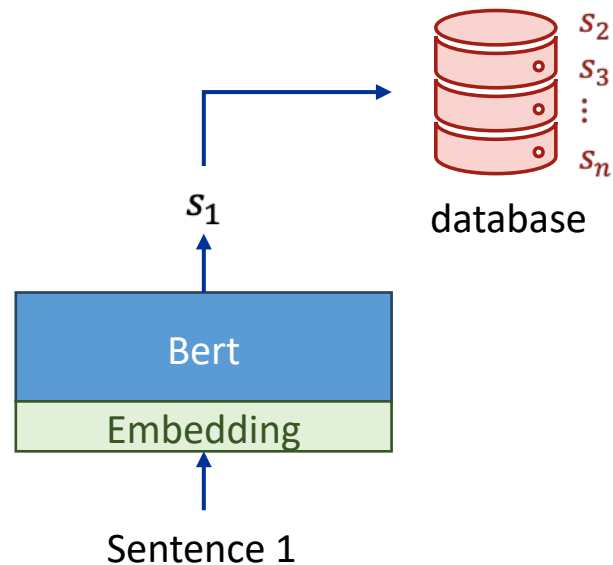
BERT generates embeddings for both sentences, and their **[CLS] embeddings** are then compared to assess similarity, typically using cosine similarity or a classifier.



Information Retrieval

Retrieving relevant documents from a large corpus or database.

The [CLS] embedding in BERT can be used as a sentence semantic representation. Therefore, a database can collect the [CLS] embedding of each sentence that needs to be retrieved, and then retrieve data by comparing their similarities.



BERT's Family

BERT's Family

There are a group of models that have been developed **inspired by the original BERT architecture**. These models aim to **improve upon BERT's performance, efficiency, or adapt it to specific tasks**. The main members of BERT's family include:

- **RoBERTa** (Robustly Optimized BERT Pretraining Approach)
- ALBERT (A Lite BERT)
- DistilBERT and TinyBERT
- XLNet: based on Transformer-XL, where XL means extra-long
- ...

RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) introduces several key **improvements over the original BERT model**, focusing on enhancing pre-training efficiency and effectiveness.

- **No NSP Task**: RoBERTa removes the NSP task. Research showed that **NSP** task did not contribute significantly to BERT's performance, so RoBERTa skips it entirely, allowing the model to focus more on the masked language modelling task.
- **More Data**: RoBERTa is trained on a significantly larger dataset than BERT and for a longer period. This helps RoBERTa improve performance on downstream tasks.
- **Optimised Hyperparameters**: RoBERTa makes **several optimisations to hyperparameters**, including learning rate schedules, to further enhance the model's performance during pre-training and fine-tuning.

ALBERT

ALBERT (A Lite BERT) focuses on **reducing the model's size** and **improving efficiency without sacrificing performance**.

- **Parameter Sharing**: ALBERT reduces the number of parameters by **sharing weights across all layers** in the model. Instead of having separate parameters for each layer, ALBERT reuses the same parameters, **significantly reducing the model size**.
- **Factorised Embedding Parameterisation**: ALBERT factorises the embedding layer into **two smaller matrices**, separating the size of the hidden layers from the size of the vocabulary embeddings.
- **Sentence Order Prediction (SOP)**: Instead of NSP, ALBERT is trained to determine whether two consecutive sentences are **in the correct order or reversed**.

DistilBERT and TinyBERT

DistilBERT and TinyBERT both use **knowledge distillation** to **significantly compress the model size** while **ensuring** as much of the **original model's performance** as possible (achieving about 97% of BERT's performance).

Knowledge Distillation: DistilBERT is trained using knowledge distillation, a technique where a smaller model (the student) learns to mimic the output of a larger, pre-trained model (the teacher, in this case, BERT).

Fewer Layers: DistilBERT reduces the number of Transformer layers by half.

TinyBERT employs a **more complex distillation process**, including layer-wise distillation and task-specific fine-tuning, to create **an even smaller and more efficient model** with strong performance on specific tasks.

XLNet

XLNet introduces several key improvements over BERT by combining the strengths of both autoregressive models and BERT's bidirectional context modelling.

Transformer-XL Architecture: XLNet builds on the Transformer-XL architecture, which is designed to **handle longer text sequences** by incorporating segment-level recurrence and relative positional encoding. This allows XLNet to capture long-range dependencies more effectively than **BERT, which is limited by fixed-length input** segments.

Permutation Language Modelling: XLNet employs a permutation-based language modelling technique to let model predict **whether word orders are switched** instead of masking words.

Autoregressive and Bidirectional Context: XLNet integrates autoregressive language modelling, where tokens are predicted based on previously seen tokens, with bidirectional context modelling.

Conclusion

Pre-trained Models: BERT is a pre-trained, bidirectional language model designed to capture context from both sides of a word.

Architecture: Built on Transformer encoders, BERT processes text using token, positional, and segment embeddings.

Training: BERT is trained using Masked Language Modelling (MLM) and Next Sentence Prediction (NSP) to develop deep language understanding.

Fine-tuning: Adapted for specific NLP tasks by adding task-specific layers and fine-tuning on smaller datasets.

BERT's Impact: BERT has inspired variants like RoBERTa, ALBERT, and DistilBERT, each improving efficiency or performance for specific tasks.